Audspect BAS — Disaster Recovery Guide

Classification: Internal — Audspect Engineering / Confidential
Platform Version: v1.7.5

Recovery Time and Point Objectives

Objective	Target
RTO (Recovery Time Objective)	< 2 hours for full platform restoration
RPO (Recovery Point Objective)	< 24 hours with daily backup; < 1 hour with hourly backup

Reference Facts

These are load-bearing for every recovery procedure below — get them wrong and a restore command silently fails or restores into the wrong database:

Fact	Value
Default listening port	`9443`
Postgres database name	`bas_platform`

Postgres user	`bas_user`
Postgres data volume	`audspect-postgres-data`
Container names	`audspect-orchestrator`, `audspect-postgres`, `audspect-caldera`, `audspect-chrome`
Compose service names	`orchestrator`, `postgres`, `caldera`, `chrome` (used with `docker compose restart <service>`)
Default install directory (`DATA_DIR`)	`/opt/audspect`
Delivery ZIP naming	`bas-install-<version>.zip`

Disaster Scenarios and Procedures

Scenario 1: Orchestrator container crash (no data loss)

Symptoms: Dashboard unreachable; `docker ps` shows `audspect-orchestrator` in Exit state.

Impact: Agents queue heartbeats locally; no runs dispatch; no result submission.

Recovery:

```
docker logs audspect-orchestrator --tail=30
sudo systemctl restart audspect 2>/dev/null \
  || (cd /opt/audspect && docker compose restart orchestr
```

Expected recovery time: **<5 minutes**. Agents reconnect automatically.

Scenario 2: PostgreSQL container crash (no data loss — volume intact)

Symptoms: Orchestrator logs show database connection errors; `/health` returns `{"db":"unavailable"}`.

Impact: No data loss. All data in the `audspect-postgres-data` Docker volume.

Recovery:

```
cd /opt/audspect
docker compose restart postgres
# Wait for postgres to become healthy, then reconnect the
docker compose restart orchestrator
curl http://localhost:9443/health     # Verify
```

Expected recovery time: **<5 minutes**.

Scenario 3: Full server failure (OS/hardware failure, Docker volumes intact)

Symptoms: Server completely unreachable.

If volumes survived (same hardware with OS reinstall):

1. Reinstall Ubuntu and Docker
2. Transfer the delivery ZIP for the currently-installed version
3. Run `install.sh --install --config setup.conf` using the **same `DATA_DIR`** as before — Postgres will pick up the existing `audspect-postgres-data` volume automatically as long as it wasn't removed

```
cd /opt
sudo unzip bas-install-<version>.zip && cd bas-install-<v
```

```
sudo bash install.sh --install --config setup.conf --yes
curl http://localhost:9443/health
```

Do not hand-reconstruct `.env` or run raw `docker compose up` here — `install.sh` is what generates/preserves secrets consistently; a mismatched `POSTGRES_PASSWORD` against the surviving volume's actual password is exactly the crash-loop this procedure exists to avoid. If you have a backed-up `setup.conf` with the original `DB_PASSWORD` matching the surviving volume, use it.

Expected recovery time: **30–60 minutes**.

Scenario 4: Full server failure with volume loss (bare metal failure)

Symptoms: Server physically dead; storage unrecoverable.

Recovery from SQL backup:

1. Provision a new Ubuntu server, install Docker
2. Transfer the delivery ZIP, run `install.sh --install --config setup.conf` fresh (new empty volume gets created)
3. Stop the stack, restore the SQL dump into the fresh database, restart

```
cd /opt
sudo unzip bas-install-<version>.zip && cd bas-install-<v
sudo bash install.sh --install --config setup.conf --yes

# Stop the orchestrator so it isn't writing during restor
sudo systemctl stop audspect 2>/dev/null || (cd /opt/auds

# Restore from backup (overwrites the freshly-created emp
zcat /backup/bas-YYYYMMDD.sql.gz | \
  docker exec -i audspect-postgres psql -U bas_user -d ba

# Restart everything
```

```
sudo systemctl start audspect 2>/dev/null || (cd /opt/aud
curl http://localhost:9443/health
```

Expected recovery time: **1–2 hours** (including data restore).

Scenario 5: Accidental data deletion

Affected data: Scenario runs, findings, agents, users — accidentally deleted via API or direct SQL.

Recovery options:

Option A: Full restore (loses all data after backup):

1. Stop the orchestrator
2. Drop and recreate the `bas_platform` database
3. Restore from backup
4. Restart the orchestrator

Option B: Surgical recovery (preserve recent data):

1. Spin up a temporary recovery PostgreSQL instance on a different port
2. Restore backup into the recovery instance
3. Export only the deleted rows using `COPY ... TO STDOUT`
4. Insert into production

```
# Spin up recovery instance
docker run -d --name bas-recovery -p 5433:5432 \
  -e POSTGRES_USER=bas_user -e POSTGRES_PASSWORD=recovery
  postgres:16-alpine

sleep 10

# Restore backup into recovery instance
zcat /backup/bas-YYYYMMDD.sql.gz | \
  docker exec -i bas-recovery psql -U bas_user -d bas_pla
```

```
# Extract deleted finding rows
docker exec bas-recovery psql -U bas_user -d bas_platform
  -c "\COPY (SELECT * FROM findings WHERE id IN ('find-ab

# (Transfer CSV from recovery container, insert into prod

# Cleanup
docker stop bas-recovery && docker rm bas-recovery
```

Scenario 6: Content-signing key lost (`orchestrator/private_key.pem`)

Symptoms: `windows-build.ps1` step 0b fails to sign scenarios, or a freshly-built orchestrator refuses to load its own bundled scenarios ("signature invalid").

Impact: Cannot release a new build that re-signs scenarios/the manifest under the existing key lineage. Already-deployed customer installs are unaffected — their running binary already has the matching public key embedded and their scenarios already verify.

Recovery: This key has no GPG import/export path — it's a plain PKCS1 PEM written by `go run orchestrator/scripts/signer.go keygen`. If a backup exists, just restore the file to `orchestrator/private_key.pem`. If no backup exists, it's unrecoverable: generate a new keypair, update `ScenarioPublicKeyPEM` in `internal/integrity/signing.go`, and ship a new orchestrator release that re-signs everything. See Signing Infrastructure — this is the **content-signing** key, not the GPG bundle-signing key (Scenario 6b).

Prevention: Always maintain an off-site backup of `orchestrator/private_key.pem`.

Scenario 6b: GPG bundle-signing key lost or

compromised

Symptoms: `windows-build.ps1` step 9b can't find the `releases@audspect.com` key; delivery ZIPs ship unsigned (with a build warning, not a failure).

Impact: New delivery ZIPs can't be authenticated by customers until re-signed. This is a **separate key** from Scenario 6 above — it only protects the ZIP's transport integrity, not scenario/manifest content, and losing it does not affect any already-deployed install's ability to verify scenarios.

Recovery: If a backup exists, `gpg --import audspect-signing-key-private.asc`. If no backup, generate a new key, publish the new `pubkey.asc`, and treat as a rotation (see Signing Infrastructure → Key Rotation Procedure). If compromise (not just loss) is suspected, also revoke the old key and notify customers to re-verify their most recent download.

Prevention: Off-site backup of the GPG private key export.

Scenario 7: License file lost

Symptoms: Platform starts with a warning about a missing license; new scenario dispatches blocked.

Recovery: Contact Audspect support with the Customer ID. License files can be re-issued. If you have a backup of the license file, the Customer ID is readable from its header:

```
head -5 bas.lic
```

If no backup of the license: contact support with your organization name and original purchase details.

Recovery Priority Matrix

Component	Priority	RTO
PostgreSQL container restart	P1 (immediate)	5 min
Orchestrator container restart	P1 (immediate)	5 min
Full server restore (volume intact)	P2	60 min
Full restore from SQL backup	P2	2 hours
Content-signing key recovery from backup	P3	30 min
GPG bundle-signing key recovery from backup	P3	30 min
New signing key generation + release (either key)	P3	1–2 days
License re-issue	P3	4–24 hours (support ticket)

Runbook Contact List

Situation	Contact
License re-issue needed	support@audspect.com
Database corruption suspected	engineering@audspect.com
Signing key compromise (either key)	security@audspect.com (treat as incident)
Physical hardware failure	customer's IT operations team

© Audspect Engineering — Internal / Confidential